
Stream:	Internet Engineering Task Force (IETF)		
RFC:	9110		
STD:	97		
Obsoletes:	2818 , 7230 , 7231 , 7232 , 7233 , 7235 , 7538 , 7615 , 7694		
Updates:	3864		
Category:	Standards Track		
Published:	June 2022		
ISSN:	2070-1721		
Authors:	R. Fielding, Ed. <i>Adobe</i>	M. Nottingham, Ed. <i>Fastly</i>	J. Reschke, Ed. <i>greenbytes</i>

RFC 9110

HTTP Semantics

Abstract

The Hypertext Transfer Protocol (HTTP) is a stateless application-level protocol for distributed, collaborative, hypertext information systems. This document describes the overall architecture of HTTP, establishes common terminology, and defines aspects of the protocol that are shared by all versions. In this definition are core protocol elements, extensibility mechanisms, and the "http" and "https" Uniform Resource Identifier (URI) schemes.

This document updates RFC 3864 and obsoletes RFCs 2818, 7231, 7232, 7233, 7235, 7538, 7615, 7694, and portions of 7230.

Status of This Memo

This is an Internet Standards Track document.

This document is a product of the Internet Engineering Task Force (IETF). It represents the consensus of the IETF community. It has received public review and has been approved for publication by the Internet Engineering Steering Group (IESG). Further information on Internet Standards is available in Section 2 of RFC 7841.

Information about the current status of this document, any errata, and how to provide feedback on it may be obtained at <https://www.rfc-editor.org/info/rfc9110>.

Copyright Notice

Copyright (c) 2022 IETF Trust and the persons identified as the document authors. All rights reserved.

This document is subject to BCP 78 and the IETF Trust's Legal Provisions Relating to IETF Documents (<https://trustee.ietf.org/license-info>) in effect on the date of publication of this document. Please review these documents carefully, as they describe your rights and restrictions with respect to this document. Code Components extracted from this document must include Revised BSD License text as described in Section 4.e of the Trust Legal Provisions and are provided without warranty as described in the Revised BSD License.

This document may contain material from IETF Documents or IETF Contributions published or made publicly available before November 10, 2008. The person(s) controlling the copyright in some of this material may not have granted the IETF Trust the right to allow modifications of such material outside the IETF Standards Process. Without obtaining an adequate license from the person(s) controlling the copyright in such materials, this document may not be modified outside the IETF Standards Process, and derivative works of it may not be created outside the IETF Standards Process, except to format it for publication as an RFC or to translate it into languages other than English.

Table of Contents

- 1. [Introduction](#)
 - 1.1. [Purpose](#)
 - 1.2. [History and Evolution](#)
 - 1.3. [Core Semantics](#)
 - 1.4. [Specifications Obsoleted by This Document](#)
- 2. [Conformance](#)
 - 2.1. [Syntax Notation](#)
 - 2.2. [Requirements Notation](#)
 - 2.3. [Length Requirements](#)
 - 2.4. [Error Handling](#)
 - 2.5. [Protocol Version](#)
- 3. [Terminology and Core Concepts](#)
 - 3.1. [Resources](#)
 - 3.2. [Representations](#)
 - 3.3. [Connections, Clients, and Servers](#)
 - 3.4. [Messages](#)
 - 3.5. [User Agents](#)
 - 3.6. [Origin Server](#)

- 3.7. Intermediaries
- 3.8. Caches
- 3.9. Example Message Exchange
- 4. Identifiers in HTTP
 - 4.1. URI References
 - 4.2. HTTP-Related URI Schemes
 - 4.2.1. http URI Scheme
 - 4.2.2. https URI Scheme
 - 4.2.3. http(s) Normalization and Comparison
 - 4.2.4. Deprecation of userinfo in http(s) URIs
 - 4.2.5. http(s) References with Fragment Identifiers
 - 4.3. Authoritative Access
 - 4.3.1. URI Origin
 - 4.3.2. http Origins
 - 4.3.3. https Origins
 - 4.3.4. https Certificate Verification
 - 4.3.5. IP-ID Reference Identity
- 5. Fields
 - 5.1. Field Names
 - 5.2. Field Lines and Combined Field Value
 - 5.3. Field Order
 - 5.4. Field Limits
 - 5.5. Field Values
 - 5.6. Common Rules for Defining Field Values
 - 5.6.1. Lists (#rule ABNF Extension)
 - 5.6.1.1. Sender Requirements
 - 5.6.1.2. Recipient Requirements
 - 5.6.2. Tokens
 - 5.6.3. Whitespace
 - 5.6.4. Quoted Strings

5.6.5. Comments

5.6.6. Parameters

5.6.7. Date/Time Formats

6. Message Abstraction

6.1. Framing and Completeness

6.2. Control Data

6.3. Header Fields

6.4. Content

6.4.1. Content Semantics

6.4.2. Identifying Content

6.5. Trailer Fields

6.5.1. Limitations on Use of Trailers

6.5.2. Processing Trailer Fields

6.6. Message Metadata

6.6.1. Date

6.6.2. Trailer

7. Routing HTTP Messages

7.1. Determining the Target Resource

7.2. Host and :authority

7.3. Routing Inbound Requests

7.3.1. To a Cache

7.3.2. To a Proxy

7.3.3. To the Origin

7.4. Rejecting Misdirected Requests

7.5. Response Correlation

7.6. Message Forwarding

7.6.1. Connection

7.6.2. Max-Forwards

7.6.3. Via

7.7. Message Transformations

7.8. Upgrade

8. Representation Data and Metadata

8.1. Representation Data

8.2. Representation Metadata

8.3. Content-Type

8.3.1. Media Type

8.3.2. Charset

8.3.3. Multipart Types

8.4. Content-Encoding

8.4.1. Content Codings

8.4.1.1. Compress Coding

8.4.1.2. Deflate Coding

8.4.1.3. Gzip Coding

8.5. Content-Language

8.5.1. Language Tags

8.6. Content-Length

8.7. Content-Location

8.8. Validator Fields

8.8.1. Weak versus Strong

8.8.2. Last-Modified

8.8.2.1. Generation

8.8.2.2. Comparison

8.8.3. ETag

8.8.3.1. Generation

8.8.3.2. Comparison

8.8.3.3. Example: Entity Tags Varying on Content-Negotiated Resources

9. Methods

9.1. Overview

9.2. Common Method Properties

9.2.1. Safe Methods

9.2.2. Idempotent Methods

9.2.3. Methods and Caching

9.3. Method Definitions

9.3.1. GET

9.3.2. HEAD

9.3.3. POST

9.3.4. PUT

9.3.5. DELETE

9.3.6. CONNECT

9.3.7. OPTIONS

9.3.8. TRACE

10. Message Context

10.1. Request Context Fields

10.1.1. Expect

10.1.2. From

10.1.3. Referer

10.1.4. TE

10.1.5. User-Agent

10.2. Response Context Fields

10.2.1. Allow

10.2.2. Location

10.2.3. Retry-After

10.2.4. Server

11. HTTP Authentication

11.1. Authentication Scheme

11.2. Authentication Parameters

11.3. Challenge and Response

11.4. Credentials

11.5. Establishing a Protection Space (Realm)

11.6. Authenticating Users to Origin Servers

11.6.1. WWW-Authenticate

11.6.2. Authorization

11.6.3. Authentication-Info

11.7. Authenticating Clients to Proxies

11.7.1. Proxy-Authenticate

11.7.2. Proxy-Authorization

11.7.3. Proxy-Authentication-Info

12. Content Negotiation

12.1. Proactive Negotiation

12.2. Reactive Negotiation

12.3. Request Content Negotiation

12.4. Content Negotiation Field Features

12.4.1. Absence

12.4.2. Quality Values

12.4.3. Wildcard Values

12.5. Content Negotiation Fields

12.5.1. Accept

12.5.2. Accept-Charset

12.5.3. Accept-Encoding

12.5.4. Accept-Language

12.5.5. Vary

13. Conditional Requests

13.1. Preconditions

13.1.1. If-Match

13.1.2. If-None-Match

13.1.3. If-Modified-Since

13.1.4. If-Unmodified-Since

13.1.5. If-Range

13.2. Evaluation of Preconditions

13.2.1. When to Evaluate

13.2.2. Precedence of Preconditions

14. Range Requests

14.1. Range Units

14.1.1. Range Specifiers

14.1.2. Byte Ranges

14.2. Range

14.3. Accept-Ranges

14.4. Content-Range

14.5. Partial PUT

14.6. Media Type multipart/byteranges

15. Status Codes

15.1. Overview of Status Codes

15.2. Informational 1xx

15.2.1. 100 Continue

15.2.2. 101 Switching Protocols

15.3. Successful 2xx

15.3.1. 200 OK

15.3.2. 201 Created

15.3.3. 202 Accepted

15.3.4. 203 Non-Authoritative Information

15.3.5. 204 No Content

15.3.6. 205 Reset Content

15.3.7. 206 Partial Content

15.3.7.1. Single Part

15.3.7.2. Multiple Parts

15.3.7.3. Combining Parts

15.4. Redirection 3xx

15.4.1. 300 Multiple Choices

15.4.2. 301 Moved Permanently

15.4.3. 302 Found

15.4.4. 303 See Other

15.4.5. 304 Not Modified

15.4.6. 305 Use Proxy

15.4.7. 306 (Unused)

15.4.8. 307 Temporary Redirect

15.4.9. 308 Permanent Redirect

15.5. Client Error 4xx

15.5.1. 400 Bad Request

15.5.2. 401 Unauthorized

15.5.3. 402 Payment Required

15.5.4. 403 Forbidden

15.5.5. 404 Not Found

15.5.6. 405 Method Not Allowed

15.5.7. 406 Not Acceptable

15.5.8. 407 Proxy Authentication Required

15.5.9. 408 Request Timeout

15.5.10. 409 Conflict

15.5.11. 410 Gone

15.5.12. 411 Length Required

15.5.13. 412 Precondition Failed

15.5.14. 413 Content Too Large

15.5.15. 414 URI Too Long

15.5.16. 415 Unsupported Media Type

15.5.17. 416 Range Not Satisfiable

15.5.18. 417 Expectation Failed

15.5.19. 418 (Unused)

15.5.20. 421 Misdirected Request

15.5.21. 422 Unprocessable Content

15.5.22. 426 Upgrade Required

15.6. Server Error 5xx

15.6.1. 500 Internal Server Error

15.6.2. 501 Not Implemented

15.6.3. 502 Bad Gateway

15.6.4. 503 Service Unavailable

15.6.5. 504 Gateway Timeout

15.6.6. 505 HTTP Version Not Supported

16. Extending HTTP

16.1. Method Extensibility

16.1.1. Method Registry

16.1.2. Considerations for New Methods

16.2. Status Code Extensibility

16.2.1. Status Code Registry

16.2.2. Considerations for New Status Codes

16.3. Field Extensibility

16.3.1. Field Name Registry

16.3.2. Considerations for New Fields

16.3.2.1. Considerations for New Field Names

16.3.2.2. Considerations for New Field Values

16.4. Authentication Scheme Extensibility

16.4.1. Authentication Scheme Registry

16.4.2. Considerations for New Authentication Schemes

16.5. Range Unit Extensibility

16.5.1. Range Unit Registry

16.5.2. Considerations for New Range Units

16.6. Content Coding Extensibility

16.6.1. Content Coding Registry

16.6.2. Considerations for New Content Codings

16.7. Upgrade Token Registry

17. Security Considerations

17.1. Establishing Authority

17.2. Risks of Intermediaries

17.3. Attacks Based on File and Path Names

17.4. Attacks Based on Command, Code, or Query Injection

17.5. Attacks via Protocol Element Length

17.6. Attacks Using Shared-Dictionary Compression

17.7. Disclosure of Personal Information

17.8. Privacy of Server Log Information

17.9. Disclosure of Sensitive Information in URIs

17.10. Application Handling of Field Names

17.11. Disclosure of Fragment after Redirects

17.12. Disclosure of Product Information

17.13. Browser Fingerprinting

17.14. Validator Retention

17.15. Denial-of-Service Attacks Using Range

17.16. Authentication Considerations

17.16.1. Confidentiality of Credentials

17.16.2. Credentials and Idle Clients

17.16.3. Protection Spaces

17.16.4. Additional Response Fields

18. IANA Considerations

18.1. URI Scheme Registration

18.2. Method Registration

18.3. Status Code Registration

18.4. Field Name Registration

18.5. Authentication Scheme Registration

18.6. Content Coding Registration

18.7. Range Unit Registration

[18.8. Media Type Registration](#)[18.9. Port Registration](#)[18.10. Upgrade Token Registration](#)[19. References](#)[19.1. Normative References](#)[19.2. Informative References](#)[Appendix A. Collected ABNF](#)[Appendix B. Changes from Previous RFCs](#)[B.1. Changes from RFC 2818](#)[B.2. Changes from RFC 7230](#)[B.3. Changes from RFC 7231](#)[B.4. Changes from RFC 7232](#)[B.5. Changes from RFC 7233](#)[B.6. Changes from RFC 7235](#)[B.7. Changes from RFC 7538](#)[B.8. Changes from RFC 7615](#)[B.9. Changes from RFC 7694](#)[Acknowledgements](#)[Index](#)[Authors' Addresses](#)

1. Introduction

1.1. Purpose

The Hypertext Transfer Protocol (HTTP) is a family of stateless, application-level, request/response protocols that share a generic interface, extensible semantics, and self-descriptive messages to enable flexible interaction with network-based hypertext information systems.

HTTP hides the details of how a service is implemented by presenting a uniform interface to clients that is independent of the types of resources provided. Likewise, servers do not need to be aware of each client's purpose: a request can be considered in isolation rather than being

associated with a specific type of client or a predetermined sequence of application steps. This allows general-purpose implementations to be used effectively in many different contexts, reduces interaction complexity, and enables independent evolution over time.

HTTP is also designed for use as an intermediation protocol, wherein proxies and gateways can translate non-HTTP information systems into a more generic interface.

One consequence of this flexibility is that the protocol cannot be defined in terms of what occurs behind the interface. Instead, we are limited to defining the syntax of communication, the intent of received communication, and the expected behavior of recipients. If the communication is considered in isolation, then successful actions ought to be reflected in corresponding changes to the observable interface provided by servers. However, since multiple clients might act in parallel and perhaps at cross-purposes, we cannot require that such changes be observable beyond the scope of a single response.

1.2. History and Evolution

HTTP has been the primary information transfer protocol for the World Wide Web since its introduction in 1990. It began as a trivial mechanism for low-latency requests, with a single method (GET) to request transfer of a presumed hypertext document identified by a given pathname. As the Web grew, HTTP was extended to enclose requests and responses within messages, transfer arbitrary data formats using MIME-like media types, and route requests through intermediaries. These protocols were eventually defined as HTTP/0.9 and HTTP/1.0 (see [\[HTTP/1.0\]](#)).

HTTP/1.1 was designed to refine the protocol's features while retaining compatibility with the existing text-based messaging syntax, improving its interoperability, scalability, and robustness across the Internet. This included length-based data delimiters for both fixed and dynamic (chunked) content, a consistent framework for content negotiation, opaque validators for conditional requests, cache controls for better cache consistency, range requests for partial updates, and default persistent connections. HTTP/1.1 was introduced in 1995 and published on the Standards Track in 1997 [[RFC2068](#)], revised in 1999 [[RFC2616](#)], and revised again in 2014 ([[RFC7230](#)] through [[RFC7235](#)]).

HTTP/2 ([\[HTTP/2\]](#)) introduced a multiplexed session layer on top of the existing TLS and TCP protocols for exchanging concurrent HTTP messages with efficient field compression and server push. HTTP/3 ([\[HTTP/3\]](#)) provides greater independence for concurrent messages by using QUIC as a secure multiplexed transport over UDP instead of TCP.

All three major versions of HTTP rely on the semantics defined by this document. They have not obsoleted each other because each one has specific benefits and limitations depending on the context of use. Implementations are expected to choose the most appropriate transport and messaging syntax for their particular context.

This revision of HTTP separates the definition of semantics (this document) and caching ([\[CACHING\]](#)) from the current HTTP/1.1 messaging syntax ([\[HTTP/1.1\]](#)) to allow each major protocol version to progress independently while referring to the same core semantics.

1.3. Core Semantics

HTTP provides a uniform interface for interacting with a resource ([Section 3.1](#)) – regardless of its type, nature, or implementation – by sending messages that manipulate or transfer representations ([Section 3.2](#)).

Each message is either a request or a response. A client constructs request messages that communicate its intentions and routes those messages toward an identified origin server. A server listens for requests, parses each message received, interprets the message semantics in relation to the identified target resource, and responds to that request with one or more response messages. The client examines received responses to see if its intentions were carried out, determining what to do next based on the status codes and content received.

HTTP semantics include the intentions defined by each request method ([Section 9](#)), extensions to those semantics that might be described in request header fields, status codes that describe the response ([Section 15](#)), and other control data and resource metadata that might be given in response fields.

Semantics also include representation metadata that describe how content is intended to be interpreted by a recipient, request header fields that might influence content selection, and the various selection algorithms that are collectively referred to as "content negotiation" ([Section 12](#)).

1.4. Specifications Obsoleted by This Document

Title	Reference	See
HTTP Over TLS	[RFC2818]	B.1
HTTP/1.1 Message Syntax and Routing [*]	[RFC7230]	B.2
HTTP/1.1 Semantics and Content	[RFC7231]	B.3
HTTP/1.1 Conditional Requests	[RFC7232]	B.4
HTTP/1.1 Range Requests	[RFC7233]	B.5
HTTP/1.1 Authentication	[RFC7235]	B.6
HTTP Status Code 308 (Permanent Redirect)	[RFC7538]	B.7
HTTP Authentication-Info and Proxy-Authentication-Info Response Header Fields	[RFC7615]	B.8
HTTP Client-Initiated Content-Encoding	[RFC7694]	B.9

Table 1

[*] This document only obsoletes the portions of [RFC 7230](#) that are independent of the HTTP/1.1 messaging syntax and connection management; the remaining bits of [RFC 7230](#) are obsoleted by "HTTP/1.1" [[HTTP/1.1](#)].

2. Conformance

2.1. Syntax Notation

This specification uses the Augmented Backus-Naur Form (ABNF) notation of [[RFC5234](#)], extended with the notation for case-sensitivity in strings defined in [[RFC7405](#)].

It also uses a list extension, defined in [Section 5.6.1](#), that allows for compact definition of comma-separated lists using a "#" operator (similar to how the "*" operator indicates repetition).

[Appendix A](#) shows the collected grammar with all list operators expanded to standard ABNF notation.

As a convention, ABNF rule names prefixed with "obs-" denote obsolete grammar rules that appear for historical reasons.

The following core rules are included by reference, as defined in [Appendix B.1](#) of [[RFC5234](#)]: ALPHA (letters), CR (carriage return), CRLF (CR LF), CTL (controls), DIGIT (decimal 0-9), DQUOTE (double quote), HEXDIG (hexadecimal 0-9/A-F/a-f), HTAB (horizontal tab), LF (line feed), OCTET (any 8-bit sequence of data), SP (space), and VCHAR (any visible US-ASCII character).

[Section 5.6](#) defines some generic syntactic components for field values.

This specification uses the terms "character", "character encoding scheme", "charset", and "protocol element" as they are defined in [[RFC6365](#)].

2.2. Requirements Notation

The key words "MUST", "MUST NOT", "REQUIRED", "SHALL", "SHALL NOT", "SHOULD", "SHOULD NOT", "RECOMMENDED", "NOT RECOMMENDED", "MAY", and "OPTIONAL" in this document are to be interpreted as described in BCP 14 [[RFC2119](#)] [[RFC8174](#)] when, and only when, they appear in all capitals, as shown here.

This specification targets conformance criteria according to the role of a participant in HTTP communication. Hence, requirements are placed on senders, recipients, clients, servers, user agents, intermediaries, origin servers, proxies, gateways, or caches, depending on what behavior is being constrained by the requirement. Additional requirements are placed on implementations, resource owners, and protocol element registrations when they apply beyond the scope of a single communication.

The verb "generate" is used instead of "send" where a requirement applies only to implementations that create the protocol element, rather than an implementation that forwards a received element downstream.

An implementation is considered conformant if it complies with all of the requirements associated with the roles it partakes in HTTP.

A sender **MUST NOT** generate protocol elements that do not match the grammar defined by the corresponding ABNF rules. Within a given message, a sender **MUST NOT** generate protocol elements or syntax alternatives that are only allowed to be generated by participants in other roles (i.e., a role that the sender does not have for that message).

Conformance to HTTP includes both conformance to the particular messaging syntax of the protocol version in use and conformance to the semantics of protocol elements sent. For example, a client that claims conformance to HTTP/1.1 but fails to recognize the features required of HTTP/1.1 recipients will fail to interoperate with servers that adjust their responses in accordance with those claims. Features that reflect user choices, such as content negotiation and user-selected extensions, can impact application behavior beyond the protocol stream; sending protocol elements that inaccurately reflect a user's choices will confuse the user and inhibit choice.

When an implementation fails semantic conformance, recipients of that implementation's messages will eventually develop workarounds to adjust their behavior accordingly. A recipient **MAY** employ such workarounds while remaining conformant to this protocol if the workarounds are limited to the implementations at fault. For example, servers often scan portions of the User-Agent field value, and user agents often scan the Server field value, to adjust their own behavior with respect to known bugs or poorly chosen defaults.

2.3. Length Requirements

A recipient **SHOULD** parse a received protocol element defensively, with only marginal expectations that the element will conform to its ABNF grammar and fit within a reasonable buffer size.

HTTP does not have specific length limitations for many of its protocol elements because the lengths that might be appropriate will vary widely, depending on the deployment context and purpose of the implementation. Hence, interoperability between senders and recipients depends on shared expectations regarding what is a reasonable length for each protocol element. Furthermore, what is commonly understood to be a reasonable length for some protocol elements has changed over the course of the past three decades of HTTP use and is expected to continue changing in the future.

At a minimum, a recipient **MUST** be able to parse and process protocol element lengths that are at least as long as the values that it generates for those same protocol elements in other messages. For example, an origin server that publishes very long URI references to its own resources needs to be able to parse and process those same references when received as a target URI.

Many received protocol elements are only parsed to the extent necessary to identify and forward that element downstream. For example, an intermediary might parse a received field into its field name and field value components, but then forward the field without further parsing inside the field value.

2.4. Error Handling

A recipient **MUST** interpret a received protocol element according to the semantics defined for it by this specification, including extensions to this specification, unless the recipient has determined (through experience or configuration) that the sender incorrectly implements what is implied by those semantics. For example, an origin server might disregard the contents of a received [Accept-Encoding](#) header field if inspection of the [User-Agent](#) header field indicates a specific implementation version that is known to fail on receipt of certain content codings.

Unless noted otherwise, a recipient **MAY** attempt to recover a usable protocol element from an invalid construct. HTTP does not define specific error handling mechanisms except when they have a direct impact on security, since different applications of the protocol require different error handling strategies. For example, a Web browser might wish to transparently recover from a response where the [Location](#) header field doesn't parse according to the ABNF, whereas a systems control client might consider any form of error recovery to be dangerous.

Some requests can be automatically retried by a client in the event of an underlying connection failure, as described in [Section 9.2.2](#).

2.5. Protocol Version

HTTP's version number consists of two decimal digits separated by a "." (period or decimal point). The first digit (major version) indicates the messaging syntax, whereas the second digit (minor version) indicates the highest minor version within that major version to which the sender is conformant (able to understand for future communication).

While HTTP's core semantics don't change between protocol versions, their expression "on the wire" can change, and so the HTTP version number changes when incompatible changes are made to the wire format. Additionally, HTTP allows incremental, backwards-compatible changes to be made to the protocol without changing its version through the use of defined extension points ([Section 16](#)).

The protocol version as a whole indicates the sender's conformance with the set of requirements laid out in that version's corresponding specification(s). For example, the version "HTTP/1.1" is defined by the combined specifications of this document, "HTTP Caching" [[CACHING](#)], and "HTTP/1.1" [[HTTP/1.1](#)].

HTTP's major version number is incremented when an incompatible message syntax is introduced. The minor number is incremented when changes made to the protocol have the effect of adding to the message semantics or implying additional capabilities of the sender.

The minor version advertises the sender's communication capabilities even when the sender is only using a backwards-compatible subset of the protocol, thereby letting the recipient know that more advanced features can be used in response (by servers) or in future requests (by clients).

When a major version of HTTP does not define any minor versions, the minor version "0" is implied. The "0" is used when referring to that protocol within elements that require a minor version identifier.

3. Terminology and Core Concepts

HTTP was created for the World Wide Web (WWW) architecture and has evolved over time to support the scalability needs of a worldwide hypertext system. Much of that architecture is reflected in the terminology used to define HTTP.

3.1. Resources

The target of an HTTP request is called a "resource". HTTP does not limit the nature of a resource; it merely defines an interface that might be used to interact with resources. Most resources are identified by a Uniform Resource Identifier (URI), as described in [Section 4](#).

One design goal of HTTP is to separate resource identification from request semantics, which is made possible by vesting the request semantics in the request method ([Section 9](#)) and a few request-modifying header fields. A resource cannot treat a request in a manner inconsistent with the semantics of the method of the request. For example, though the URI of a resource might imply semantics that are not safe, a client can expect the resource to avoid actions that are unsafe when processing a request with a safe method (see [Section 9.2.1](#)).

HTTP relies upon the Uniform Resource Identifier (URI) standard [[URI](#)] to indicate the target resource ([Section 7.1](#)) and relationships between resources.

3.2. Representations

A "representation" is information that is intended to reflect a past, current, or desired state of a given resource, in a format that can be readily communicated via the protocol. A representation consists of a set of representation metadata and a potentially unbounded stream of representation data ([Section 8](#)).

HTTP allows "information hiding" behind its uniform interface by defining communication with respect to a transferable representation of the resource state, rather than transferring the resource itself. This allows the resource identified by a URI to be anything, including temporal functions like "the current weather in Laguna Beach", while potentially providing information that represents that resource at the time a message is generated [[REST](#)].

The uniform interface is similar to a window through which one can observe and act upon a thing only through the communication of messages to an independent actor on the other side. A shared abstraction is needed to represent ("take the place of") the current or desired state of that thing in our communications. When a representation is hypertext, it can provide both a representation of the resource state and processing instructions that help guide the recipient's future interactions.

A [target resource](#) might be provided with, or be capable of generating, multiple representations that are each intended to reflect the resource's current state. An algorithm, usually based on [content negotiation](#) ([Section 12](#)), would be used to select one of those representations as being most applicable to a given request. This "selected representation" provides the data and metadata for evaluating conditional requests ([Section 13](#)) and constructing the content for [200 \(OK\)](#), [206 \(Partial Content\)](#), and [304 \(Not Modified\)](#) responses to GET ([Section 9.3.1](#)).

3.3. Connections, Clients, and Servers

HTTP is a client/server protocol that operates over a reliable transport- or session-layer "connection".

An HTTP "client" is a program that establishes a connection to a server for the purpose of sending one or more HTTP requests. An HTTP "server" is a program that accepts connections in order to service HTTP requests by sending HTTP responses.

The terms client and server refer only to the roles that these programs perform for a particular connection. The same program might act as a client on some connections and a server on others.

HTTP is defined as a stateless protocol, meaning that each request message's semantics can be understood in isolation, and that the relationship between connections and messages on them has no impact on the interpretation of those messages. For example, a CONNECT request ([Section 9.3.6](#)) or a request with the Upgrade header field ([Section 7.8](#)) can occur at any time, not just in the first message on a connection. Many implementations depend on HTTP's stateless design in order to reuse proxied connections or dynamically load balance requests across multiple servers.

As a result, a server **MUST NOT** assume that two requests on the same connection are from the same user agent unless the connection is secured and specific to that agent. Some non-standard HTTP extensions (e.g., [[RFC4559](#)]) have been known to violate this requirement, resulting in security and interoperability problems.

3.4. Messages

HTTP is a stateless request/response protocol for exchanging "messages" across a [connection](#). The terms "sender" and "recipient" refer to any implementation that sends or receives a given message, respectively.

A client sends requests to a server in the form of a "request" message with a method ([Section 9](#)) and request target ([Section 7.1](#)). The request might also contain header fields ([Section 6.3](#)) for request modifiers, client information, and representation metadata, content ([Section 6.4](#)) intended for processing in accordance with the method, and trailer fields ([Section 6.5](#)) to communicate information collected while sending the content.

A server responds to a client's request by sending one or more "response" messages, each including a status code ([Section 15](#)). The response might also contain header fields for server information, resource metadata, and representation metadata, content to be interpreted in accordance with the status code, and trailer fields to communicate information collected while sending the content.

3.5. User Agents

The term "user agent" refers to any of the various client programs that initiate a request.

The most familiar form of user agent is the general-purpose Web browser, but that's only a small percentage of implementations. Other common user agents include spiders (web-traversing robots), command-line tools, billboard screens, household appliances, scales, light bulbs, firmware update scripts, mobile apps, and communication devices in a multitude of shapes and sizes.

Being a user agent does not imply that there is a human user directly interacting with the software agent at the time of a request. In many cases, a user agent is installed or configured to run in the background and save its results for later inspection (or save only a subset of those results that might be interesting or erroneous). Spiders, for example, are typically given a start URI and configured to follow certain behavior while crawling the Web as a hypertext graph.

Many user agents cannot, or choose not to, make interactive suggestions to their user or provide adequate warning for security or privacy concerns. In the few cases where this specification requires reporting of errors to the user, it is acceptable for such reporting to only be observable in an error console or log file. Likewise, requirements that an automated action be confirmed by the user before proceeding might be met via advance configuration choices, run-time options, or simple avoidance of the unsafe action; confirmation does not imply any specific user interface or interruption of normal processing if the user has already made that choice.

3.6. Origin Server

The term "origin server" refers to a program that can originate authoritative responses for a given target resource.

The most familiar form of origin server are large public websites. However, like user agents being equated with browsers, it is easy to be misled into thinking that all origin servers are alike. Common origin servers also include home automation units, configurable networking components, office machines, autonomous robots, news feeds, traffic cameras, real-time ad selectors, and video-on-demand platforms.

Most HTTP communication consists of a retrieval request (GET) for a representation of some resource identified by a URI. In the simplest case, this might be accomplished via a single bidirectional connection (==) between the user agent (UA) and the origin server (O).



Figure 1

3.7. Intermediaries

HTTP enables the use of intermediaries to satisfy requests through a chain of connections. There are three common forms of HTTP "intermediary": proxy, gateway, and tunnel. In some cases, a single intermediary might act as an origin server, proxy, gateway, or tunnel, switching behavior based on the nature of each request.

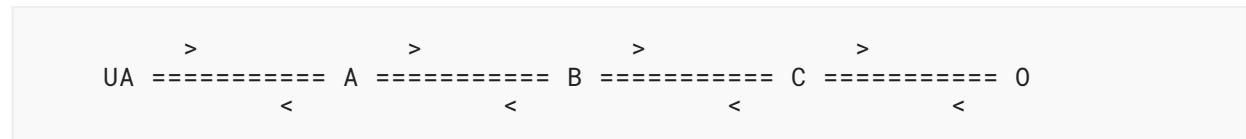


Figure 2

The figure above shows three intermediaries (A, B, and C) between the user agent and origin server. A request or response message that travels the whole chain will pass through four separate connections. Some HTTP communication options might apply only to the connection with the nearest, non-tunnel neighbor, only to the endpoints of the chain, or to all connections along the chain. Although the diagram is linear, each participant might be engaged in multiple, simultaneous communications. For example, B might be receiving requests from many clients other than A, and/or forwarding requests to servers other than C, at the same time that it is handling A's request. Likewise, later requests might be sent through a different path of connections, often based on dynamic configuration for load balancing.

The terms "upstream" and "downstream" are used to describe directional requirements in relation to the message flow: all messages flow from upstream to downstream. The terms "inbound" and "outbound" are used to describe directional requirements in relation to the request route: inbound means "toward the origin server", whereas outbound means "toward the user agent".

A "proxy" is a message-forwarding agent that is chosen by the client, usually via local configuration rules, to receive requests for some type(s) of absolute URI and attempt to satisfy those requests via translation through the HTTP interface. Some translations are minimal, such as for proxy requests for "http" URIs, whereas other requests might require translation to and from entirely different application-level protocols. Proxies are often used to group an organization's HTTP requests through a common intermediary for the sake of security services, annotation services, or shared caching. Some proxies are designed to apply transformations to selected messages or content while they are being forwarded, as described in [Section 7.7](#).

A "gateway" (a.k.a. "reverse proxy") is an intermediary that acts as an origin server for the outbound connection but translates received requests and forwards them inbound to another server or servers. Gateways are often used to encapsulate legacy or untrusted information services, to improve server performance through "accelerator" caching, and to enable partitioning or load balancing of HTTP services across multiple machines.

All HTTP requirements applicable to an origin server also apply to the outbound communication of a gateway. A gateway communicates with inbound servers using any protocol that it desires, including private extensions to HTTP that are outside the scope of this specification. However, an HTTP-to-HTTP gateway that wishes to interoperate with third-party HTTP servers needs to conform to user agent requirements on the gateway's inbound connection.

A "tunnel" acts as a blind relay between two connections without changing the messages. Once active, a tunnel is not considered a party to the HTTP communication, though the tunnel might have been initiated by an HTTP request. A tunnel ceases to exist when both ends of the relayed connection are closed. Tunnels are used to extend a virtual connection through an intermediary, such as when Transport Layer Security (TLS, [TLS13]) is used to establish confidential communication through a shared firewall proxy.

The above categories for intermediary only consider those acting as participants in the HTTP communication. There are also intermediaries that can act on lower layers of the network protocol stack, filtering or redirecting HTTP traffic without the knowledge or permission of message senders. Network intermediaries are indistinguishable (at a protocol level) from an on-path attacker, often introducing security flaws or interoperability problems due to mistakenly violating HTTP semantics.

For example, an "interception proxy" [RFC3040] (also commonly known as a "transparent proxy" [RFC1919]) differs from an HTTP proxy because it is not chosen by the client. Instead, an interception proxy filters or redirects outgoing TCP port 80 packets (and occasionally other common port traffic). Interception proxies are commonly found on public network access points, as a means of enforcing account subscription prior to allowing use of non-local Internet services, and within corporate firewalls to enforce network usage policies.

3.8. Caches

A "cache" is a local store of previous response messages and the subsystem that controls its message storage, retrieval, and deletion. A cache stores cacheable responses in order to reduce the response time and network bandwidth consumption on future, equivalent requests. Any client or server **MAY** employ a cache, though a cache cannot be used while acting as a tunnel.

The effect of a cache is that the request/response chain is shortened if one of the participants along the chain has a cached response applicable to that request. The following illustrates the resulting chain if B has a cached copy of an earlier response from O (via C) for a request that has not been cached by UA or A.

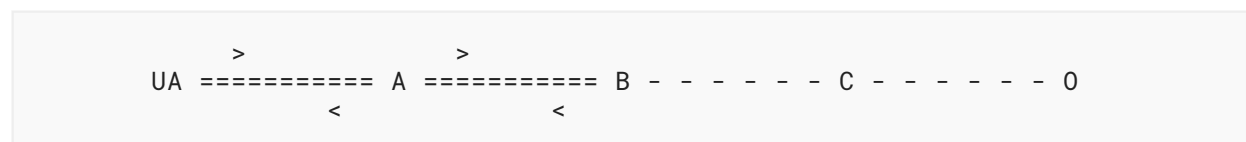


Figure 3

A response is "cacheable" if a cache is allowed to store a copy of the response message for use in answering subsequent requests. Even when a response is cacheable, there might be additional constraints placed by the client or by the origin server on when that cached response can be used for a particular request. HTTP requirements for cache behavior and cacheable responses are defined in [\[CACHING\]](#).

There is a wide variety of architectures and configurations of caches deployed across the World Wide Web and inside large organizations. These include national hierarchies of proxy caches to save bandwidth and reduce latency, content delivery networks that use gateway caching to optimize regional and global distribution of popular sites, collaborative systems that broadcast or multicast cache entries, archives of pre-fetched cache entries for use in off-line or high-latency environments, and so on.

3.9. Example Message Exchange

The following example illustrates a typical HTTP/1.1 message exchange for a GET request ([Section 9.3.1](#)) on the URI "http://www.example.com/hello.txt":

Client request:

```
GET /hello.txt HTTP/1.1
User-Agent: curl/7.64.1
Host: www.example.com
Accept-Language: en, mi
```

Server response:

```
HTTP/1.1 200 OK
Date: Mon, 27 Jul 2009 12:28:53 GMT
Server: Apache
Last-Modified: Wed, 22 Jul 2009 19:15:56 GMT
ETag: "34aa387-d-1568eb00"
Accept-Ranges: bytes
Content-Length: 51
Vary: Accept-Encoding
Content-Type: text/plain

Hello World! My content includes a trailing CRLF.
```

4. Identifiers in HTTP

Uniform Resource Identifiers (URIs) [\[URI\]](#) are used throughout HTTP as the means for identifying resources ([Section 3.1](#)).

4.1. URI References

URI references are used to target requests, indicate redirects, and define relationships.

The definitions of "URI-reference", "absolute-URI", "relative-part", "authority", "port", "host", "path-abempty", "segment", and "query" are adopted from the URI generic syntax. An "absolute-path" rule is defined for protocol elements that can contain a non-empty path component. (This rule differs slightly from the path-abempty rule of RFC 3986, which allows for an empty path, and path-absolute rule, which does not allow paths that begin with "/".) A "partial-URI" rule is defined for protocol elements that can contain a relative URI but not a fragment component.

```
URI-reference = <URI-reference, see [URI], Section 4.1>
absolute-URI  = <absolute-URI, see [URI], Section 4.3>
relative-part = <relative-part, see [URI], Section 4.2>
authority     = <authority, see [URI], Section 3.2>
uri-host      = <host, see [URI], Section 3.2.2>
port          = <port, see [URI], Section 3.2.3>
path-abempty  = <path-abempty, see [URI], Section 3.3>
segment       = <segment, see [URI], Section 3.3>
query         = <query, see [URI], Section 3.4>

absolute-path = 1*( "/" segment )
partial-URI   = relative-part [ "?" query ]
```

Each protocol element in HTTP that allows a URI reference will indicate in its ABNF production whether the element allows any form of reference (URI-reference), only a URI in absolute form (absolute-URI), only the path and optional query components (partial-URI), or some combination of the above. Unless otherwise indicated, URI references are parsed relative to the target URI (Section 7.1).

It is **RECOMMENDED** that all senders and recipients support, at a minimum, URIs with lengths of 8000 octets in protocol elements. Note that this implies some structures and on-wire representations (for example, the request line in HTTP/1.1) will necessarily be larger in some cases.

4.2. HTTP-Related URI Schemes

IANA maintains the registry of URI Schemes [BCP35] at <<https://www.iana.org/assignments/uri-schemes/>>. Although requests might target any URI scheme, the following schemes are inherent to HTTP servers:

URI Scheme	Description	Section
http	Hypertext Transfer Protocol	4.2.1
https	Hypertext Transfer Protocol Secure	4.2.2

Table 2

Note that the presence of an "http" or "https" URI does not imply that there is always an HTTP server at the identified origin listening for connections. Anyone can mint a URI, whether or not a server exists and whether or not that server currently maps that identifier to a resource. The delegated nature of registered names and IP addresses creates a federated namespace whether or not an HTTP server is present.

4.2.1. http URI Scheme

The "http" URI scheme is hereby defined for minting identifiers within the hierarchical namespace governed by a potential HTTP origin server listening for TCP ([TCP]) connections on a given port.

```
http-URI = "http" "://" authority path-abempty [ "?" query ]
```

The origin server for an "http" URI is identified by the [authority](#) component, which includes a host identifier ([URI], [Section 3.2.2](#)) and optional port number ([URI], [Section 3.2.3](#)). If the port subcomponent is empty or not given, TCP port 80 (the reserved port for WWW services) is the default. The origin determines who has the right to respond authoritatively to requests that target the identified resource, as defined in [Section 4.3.2](#).

A sender **MUST NOT** generate an "http" URI with an empty host identifier. A recipient that processes such a URI reference **MUST** reject it as invalid.

The hierarchical path component and optional query component identify the target resource within that origin server's namespace.

4.2.2. https URI Scheme

The "https" URI scheme is hereby defined for minting identifiers within the hierarchical namespace governed by a potential origin server listening for TCP connections on a given port and capable of establishing a TLS ([TLS13]) connection that has been secured for HTTP communication. In this context, "secured" specifically means that the server has been authenticated as acting on behalf of the identified authority and all HTTP communication with that server has confidentiality and integrity protection that is acceptable to both client and server.

```
https-URI = "https" "://" authority path-abempty [ "?" query ]
```

The origin server for an "https" URI is identified by the [authority](#) component, which includes a host identifier ([URI], [Section 3.2.2](#)) and optional port number ([URI], [Section 3.2.3](#)). If the port subcomponent is empty or not given, TCP port 443 (the reserved port for HTTP over TLS) is the default. The origin determines who has the right to respond authoritatively to requests that target the identified resource, as defined in [Section 4.3.3](#).

A sender **MUST NOT** generate an "https" URI with an empty host identifier. A recipient that processes such a URI reference **MUST** reject it as invalid.